

Codepop High Level Design Documentation

Version: 2

Team: SocialDrinkers (3)

- [Codepop High Level Design Documentation](#)
 - [1. Introduction](#)
 - [1.1 Security & Compliance First](#)
 - [Compliance Goals:](#)
 - [2. Hardware Platforms](#)
 - [2.1 Mobile & Touch](#)
 - [2.2 Web / Desktop](#)
 - [2.3 IoT & Future Platforms \(Optional\)](#)
 - [3. User Interface \(UI\)](#)
 - [3.1 Concept and Design](#)
 - [3.2 Color Palette](#)
 - [3.3 Framework and tools \(Tech Stack\)](#)
 - [4. Data Classification & Security](#)
 - [4.1 Definitions](#)
 - [4.2 Data Breakdown by Role](#)
 - [4.3 Encryption Guidelines](#)
 - [Data in Transit](#)
 - [Data At Rest](#)
 - [4.4 Secure Code Guidelines](#)
 - [5. System Architecture](#)
 - [5.1 Component Definitions](#)
 - [5.2 Server Subcomponents](#)
 - [5.3 Database Schema](#)
 - [6. Internal Interfaces](#)
 - [6.1 System Dataflow](#)
 - [6.2 Component Interfaces](#)
 - [7. External Interfaces](#)
 - [8. Deployment and developer workflow](#)
 - [Spin It Up Locally](#)
 - [Configuration](#)
 - [Explore the API](#)

1. Introduction

Codepop is a **distributed beverage ordering network** that lets customers order drinks from any store while keeping their account, preferences, and order history portable across locations. Behind the scenes, it's built as a robust production system with:

- **codepop_frontend**: A sleek Next.js/React application for phones, tablets, and desktops—built for speed, touchability, and seamless checkout experiences.
- **codepop_backend**: A Django 5.1 powerhouse with REST APIs, peer-to-peer server authentication, Stripe payments, and inventory orchestration across a decentralized network.

Stores run as independent servers that discover and sync with each other, customers can move between locations without re-authentication, and managers control local operations from their dashboards—all while the system keeps data consistent across the network.

1.1 Security & Compliance First

Important: This document is not legal advice. Consult legal counsel for GDPR, CCPA, HIPAA, or local data protection laws.

Compliance Goals:

- **OWASP Top 10** vulnerabilities (injection, auth bypass, XSS, etc.) are actively mitigated through Django's ORM, React's safe rendering, and CSRF protection.
- **PCI DSS Compliance:** Stripe handles all card tokenization—raw payment data never touches our servers, just secure payment references.

Details on data handling, encryption, and role-based access are in Section 4.

2. Hardware Platforms

Codepop lives in the browser—no downloads, no app store friction. Whether you're using a phone in line, a tablet behind the counter, or a desktop in the back office, the experience adapts beautifully.

2.1 Mobile & Touch

- **Adaptive Design:** Built on Next.js and Tailwind CSS, Codepop scales fluidly from iPhone screens to tablets. Touch targets are generous, spacing is natural, and scrolling feels native.
- **Smart Navigation:** Bottom tabs for quick access to home/cart/account, a slide-out menu for deeper options, drink category filters that don't clutter, and modals that feel native—all designed with thumbs in mind.
- **Cross-Platform:** Works great on Chrome, Safari, Edge, and Firefox. Whether iOS or Android, web or PWA, the experience is consistent and snappy.

2.2 Web / Desktop

- **Modern & Fast:** Next.js 16.1 + React 19.2 power a truly responsive interface. Client-side rendering keeps interactions smooth; no annoying page refreshes.
- **Battle-Tested Dependencies:** **axios** for reliable HTTP, **lucide-react** for crisp icons, Stripe's official SDK for secure payments, and Tailwind CSS for pixel-perfect design.
- **Future-Ready:** Built with PWA principles in mind—service workers and offline support can be added when you're ready.

2.3 IoT & Future Platforms (Optional)

The current codebase does not include full IoT device integration, but the architecture supports future machine telemetry and automation.

Potential extensions include:

- Real-time dispenser stock telemetry and machine alerts,
 - Predictive inventory ("this machine needs refilling in 2 hours"),
 - Gamified fulfillment (reward staff for fast machine service).
-

3. User Interface (UI)

3.1 Concept and Design

The UI is component-driven and designed to be easy to use on both mobile and web applications.

- **Reusable Components:** Shared components include `Header`, `Sidebar`, `BottomNav`, `StoreSelector`, `DrinkCard`, `CustomizeModal`, and `CategoryFilter`.
- **Role-aware Flows:** Customers get order and AI pick pages, while managers and admins access inventory, order queue, revenue, and server registry views.
- **Feedback-first Interaction:** The frontend provides loading states, error messaging, and clear prompts for checkout and chatbot actions.

3.2 Color Palette

- **Background:** #FCF8FF
- **Text:** #0A0A0A
- **Primary:** #FFFFFF
- **Secondary:** #030213
- **Accent:** #9810FA
- **Neutral:** #ECECF0

3.3 Framework and tools (Tech Stack)

Frontend:

- Next.js 16.1.7, React 19.2.3, Tailwind CSS 4, TypeScript
- HTTP: `axios`, Icons: `lucide-react`, Payments: Stripe official SDKs

Backend:

- Django 5.1.2, Django REST Framework for REST, `rest_framework_simplejwt` for auth
- OpenAPI schema (`drf_spectacular`), CORS support, custom JWT P2P validation

Data:

- PostgreSQL with Django ORM—no raw SQL, migrations handle schema evolution

Intelligence:

- **Chatbot:** `microsoft/DialoGPT-medium` for conversational drink advice

- **Recommendations:** Similarity matching over CSV-based drink attributes (flavor, caffeine, sugar, etc.) using `scikit-learn` and `pandas`.

4. Data Classification & Security

Codepop treats data with respect and rigor. Here's how we categorize what we hold:

4.1 Definitions

- **Personal:** Username, email, name, favorite store—data that identifies you
- **Sensitive:** Stripe payment references, server registry keys, machine telemetry—data that could harm if leaked
- **Operational:** Drink recipes, order history, flavor preferences, inventory snapshots, notifications—data that powers the service

4.2 Data Breakdown by Role

User Role	Personal Data Collected	Sensitive Data Collected	Other Data
Guest Customer	None	Stripe token references	Session drink/cart data
Registered Customer	Name, email, username	Stripe token references, home server	Preferences, favorites, order history
Store Manager	Name, employee meta	Local inventory and machine metrics	Store orders, notifications
Logistics Manager	Name, employee meta	Regional stock and transfer data	Inventory reports (Note: Feature non-functional)
Repair Staff	Name, employee meta	Machine health logs	Repair ticket activity (Note: Feature cut)
Admin	Name, employee meta	Access logs, user management data	Local system configuration
Super Admin	Name, employee meta	Global registry, peer server data	Global inventory and revenue

4.3 Encryption Guidelines

Data in Transit

- Client-server connections must use HTTPS/TLS.
- The API uses bearer JWT tokens and asymmetric P2P validation for cross-server authentication.
- Stripe checkout is tokenized so card data never reaches backend storage.

Data At Rest

- **Passwords:** Hashed and salted through Django's crypto—even we can't read them.

- **Payment References:** Only Stripe token IDs and metadata live in the database; actual cards are in Stripe's fortress.
- **Sensitive Fields:** Can be encrypted in the database for extra paranoia (future enhancement).

4.4 Secure Code Guidelines

- **SQL Injection:** Django ORM parameterizes all queries—attackers can't inject malicious SQL.
 - **XSS (Cross-Site Scripting):** React escapes HTML by default; text stays text, scripts stay inert. No `dangerouslySetInnerHTML` with user input.
 - **CSRF (Cross-Site Request Forgery):** Django middleware validates token origin for state-changing requests.
 - **Forbidden APIs:** No `eval()`, no raw HTML from untrusted sources, no secrets in frontend code.
 - **Error Messages:** Users see friendly, helpful errors ("Oops, try again in a moment"). Developers see detailed logs with stack traces and context.
-

5. System Architecture

Codepop's strength comes from **connected independence**: stores operate autonomously, yet customers and data flow seamlessly between them.

5.1 Component Definitions

- **Client:** Next.js frontend handles ordering, store selection, drink customization, checkout, and support chat.
- **Server:** Django backend manages authentication, order processing, inventory, revenue, notifications, AI, and P2P syncing.
- **Database:** PostgreSQL stores persistent data for users, drinks, orders, inventory, servers, and notifications.

5.2 Server Subcomponents

- **API Views:** Endpoints are defined in `backend/urls.py` and include authentication, drinks, inventory, orders, revenue, leaderboard, chatbot, AI generation, and P2P sync routes.
- **Authentication:** `backend.authentication.P2PJWTAuthentication` verifies server-issued JWTs using the public keys stored in `ServerRegistry`.
- **Payment Processing:** Stripe PaymentIntent and webhook endpoints handle checkout and payment confirmation. Supports a Mock Stripe implementation for development.
- **Peer Sync:** P2P endpoints such as `/p2p/discover/`, `/p2p/join/`, `/p2p/update-peer/`, and `/sync/masterlist/` let servers discover and synchronize across the network.

5.3 Database Schema

Key models include:

- **CustomUser:** custom user entity with role type and home server.
- **ServerRegistry:** networked store server metadata and public keys.
- **MasterList:** roaming user registry for cross-server login.
- **Drink:** drink recipes, pricing, favorite relationships, and user-created status.

- **Order**: order lifecycle, payment status, pickup time, locker combo, and Stripe IDs.
 - **Inventory**: stock quantities, thresholds, and item types.
 - **Notification**: user and global notifications.
 - **Preference**: user flavor preferences.
-

6. Internal Interfaces

6.1 System Dataflow

1. **Pick a Store**: Frontend detects your location via local Haversine calculations or you choose manually from the active server list.
2. **Load Menu**: Browser downloads drinks and inventory from that store's backend.
3. **Sign In (or Guest)**: Stripe tokenizes your payment method; local JWT grants access to your account and history.
4. **Customize & Order**: Pick a drink, tweak it, tap *Order Now*.
5. **Payment**: Stripe (or Mock Stripe) captures the charge; webhook confirms it instantly.
6. **Fulfillment**: Order queued at the store, inventory adjusted, notification sent ("Your drink is ready").
7. **You Arrive**: Show the code on your phone; staff hand it over.
8. **Chat & Rate**: Ask the AI for recommendations next time or rate the drink.

6.2 Component Interfaces

- **Client Interface**: JSON API calls under `/backend/`, bearer auth headers, and stateful front-end contexts.
 - **Server Interface**: RESTful object endpoints, P2P sync endpoints, and OpenAPI schema.
 - **Database Interface**: Django ORM models, migrations, transactions, and indexed queries.
-

7. External Interfaces

- **Stripe**: The payment backbone. We hand off card data; they secure it, process charges, handle fraud detection, and send confirmations back. Includes a local Mock Stripe implementation for seamless development testing without live API keys.
 - **AI Companions**:
 - **Chatbot**: **DialoGPT-medium** runs locally on the backend using **transformers** and **torch** (no external API calls), ensuring conversations are instant, private, and free to scale.
 - **Recommendations**: Similarity matching (e.g., "find drinks with similar flavor profiles to what you liked") uses local CSV datasets parsed via **scikit-learn** and **pandas**.
 - **Geolocation**: Local Haversine formulas and heuristics are used for distance and drive-time calculations.
 - **SSO (Future)**: Easy to bolt on Google/Microsoft/GitHub logins if you want one-click onboarding; today we use local JWT for simplicity and control.
-

8. Deployment and developer workflow

Spin It Up Locally

The entire multi-store network is orchestrated via Docker Compose.

Frontend: Server starts on <http://localhost:4000> (proxy). Hot reload on file change.

Backend: Docker Compose launches up to 9 simultaneous store instances on unique ports (e.g., [8000-8008](#)), each connected to its own isolated PostgreSQL database.

Use the provided scripts (e.g., [run_local.py](#)) to stand up the Docker containers for the databases, the frontend, and the backend instances.

Configuration

The backend reads from environment variables ([.env](#) files) for database connections and Stripe keys. For local dev, P2P signing keys fall back to PEM files at [/data/node_key.pem](#) (private) and [/data/node_key_pub.pem](#) (public).

Explore the API

Hit the interactive docs for any specific store instance (e.g., store 1 on port 8000):

- **Swagger UI:** <http://localhost:8000/api/schema/swagger-ui/>
- **ReDoc:** <http://localhost:8000/api/schema/redoc/>
- **Raw OpenAPI:** <http://localhost:8000/api/schema/>